

# Advanced Dynamic Programming Study Guide

## Advanced Dynamic Programming Study Guide

### 1. Overview

Advanced Dynamic Programming (DP) extends beyond basic tabulation and memoization into **state compression**, **interval DP**, **digit DP**, **tree DP**, **bitmask DP**, and **optimization techniques** such as **Knuth Optimization** and **Divide & Conquer Optimization**.

These patterns appear frequently in competitive programming and hard LeetCode problems.

---

### 2. Key Advanced DP Categories

#### 2.1 [Bitmask DP](#)

Used when the state depends on subsets of items.

Common problems:

- Traveling Salesman Problem (TSP)
- Assignments / matching
- Counting subsets with constraints

State example:

```
dp[mask][i] = min cost to reach subset mask ending at i
```

Mask represents which nodes/items are used.

---

#### 2.2 [Interval DP](#)

Used when the solution depends on choosing partitions or merging intervals.

Common problems:

- Burst Balloons
- Matrix Chain Multiplication
- Palindromic partitions

State example:

```
dp[l][r] = best answer for interval [l, r]
```

Transitions iterate over all possible partitions k.

---

#### 2.3 [Digit DP](#)

Used for counting numbers with constraints.

Common problems:

- Count numbers with certain digits
- Count numbers without repeated digits
- Count numbers  $\leq N$  satisfying property

State example:

```
dp[pos][tight][state] = count of valid numbers
```

---

## 2.4 [Tree DP](#)

Used when the structure is a tree.

Common problems:

- Maximum path sum
- Tree diameter
- Counting subtrees
- DP on rooted trees

State example:

```
dp[node] = best value for subtree rooted at node
```

---

## 2.5 [Graph DP](#)

Used when states depend on graph traversal.

Common problems:

- Longest path in DAG
- Counting paths
- DP on topological order

State example:

```
dp[u] = best value ending at node u
```

---

# 3. Optimization Techniques

## 3.1 [Space Optimization](#)

Reduce memory by using rolling arrays.

Example:

```
dp[i][j] depends only on dp[i-1][*]
```

→ compress to 1D.

---

### 3.2 [Monotonic Queue Optimization](#)

Used for sliding window DP transitions.

Example:

```
dp[i] = max(dp[j] + cost[j][i]) for j in window
```

---

### 3.3 [Knuth Optimization](#)

Used for DP of form:

```
dp[i][j] = min(dp[i][k] + dp[k][j])
```

with quadrangle inequality.

Reduces  $O(n^3) \rightarrow O(n^2)$ .

---

### 3.4 [Divide & Conquer Optimization](#)

Used for DP of form:

```
dp[i][j] = min over k < j
```

with monotonicity of optimal k.

Reduces  $O(n^2) \rightarrow O(n \log n)$ .

---

## 4. Example: Burst Balloons (Interval DP)

**Problem:** Max coins by bursting balloons in optimal order.

State:

```
dp[l][r] = max coins from interval (l, r)
```

Transition:

```
for k in (l+1, r):
```

```
    dp[l][r] = max(dp[l][r], dp[l][k] + dp[k][r] + nums[l]*nums[k]*nums[r])
```

---

## 5. Example: Traveling Salesman (Bitmask DP)

```
dp[mask][i] = min cost to visit all nodes in mask ending at i
```

Transition:

```
dp[mask][i] = min(dp[mask ^ (1<<i)][j] + cost[j][i])
```

---

## 6. Common Pitfalls

- Incorrect mask transitions
- Forgetting base cases in interval DP
- Mismanaging tight constraints in digit DP

- Not rooting trees before DP
  - Using DP on cyclic graphs without topological sorting
- 

## 7. Practice Roadmap

### Intermediate

- Longest Path in DAG
- Matrix Chain Multiplication
- Palindrome Partitioning II

### Advanced

- Burst Balloons
- TSP (Bitmask DP)
- Count Numbers with Unique Digits (Digit DP)
- House Robber III (Tree DP)

### Expert

- DP with Knuth Optimization
  - DP with Divide & Conquer Optimization
  - Hard interval merging problems
- 

## 8. Key Takeaways

- Advanced DP requires recognizing **structure**: intervals, trees, digits, subsets.
  - Optimization techniques reduce complexity dramatically.
  - Many hard problems reduce to a known DP pattern.
- 

End of document.